Technische Universität Berlin

Distributed Systems
Chapter 3: Architecture Models

FG Verteilte Systeme und Betriebssysteme | Distributed Systems

Agenda

3.1 Client / Server Systems

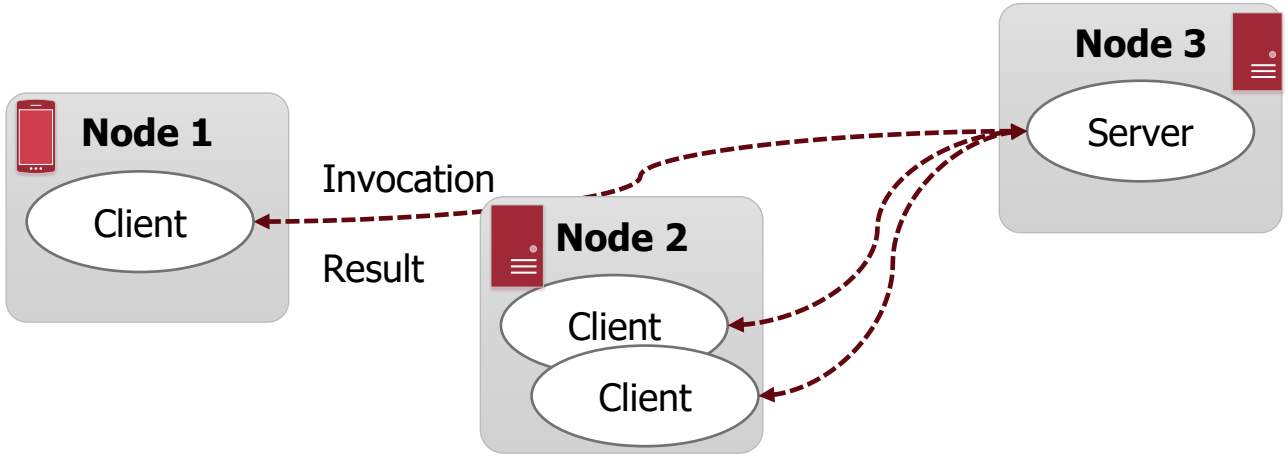
3.2 Peer-to-Peer Systems

3.3 Publish / Subscribe Systems

3.1 Client / Server Systems

Architectural model: Client/server systems are a common approach for the distribution of roles and responsibilities

- Separation of responsibilities between system components (applications, servers, other processes) is an obvious aspect in DS design
- Note that the role assignment regards the process, not the node!



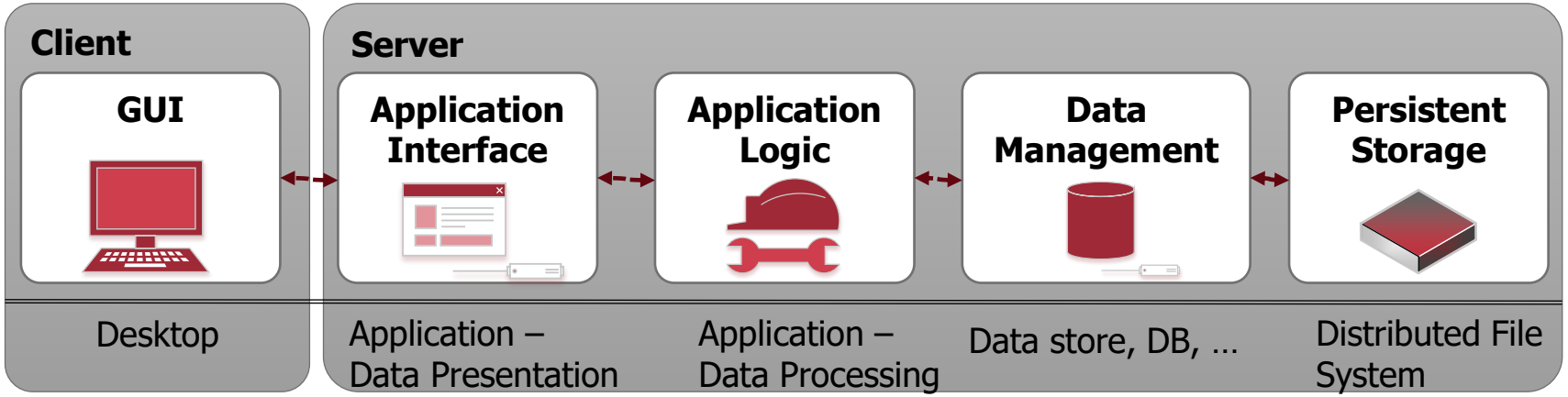
Client / Server Systems – Basic components

Desktop	Application – Data Presentation	Application – Data Processing	Data store, DB, …	Distributed File System

Client / Server Systems – Two Tier Model

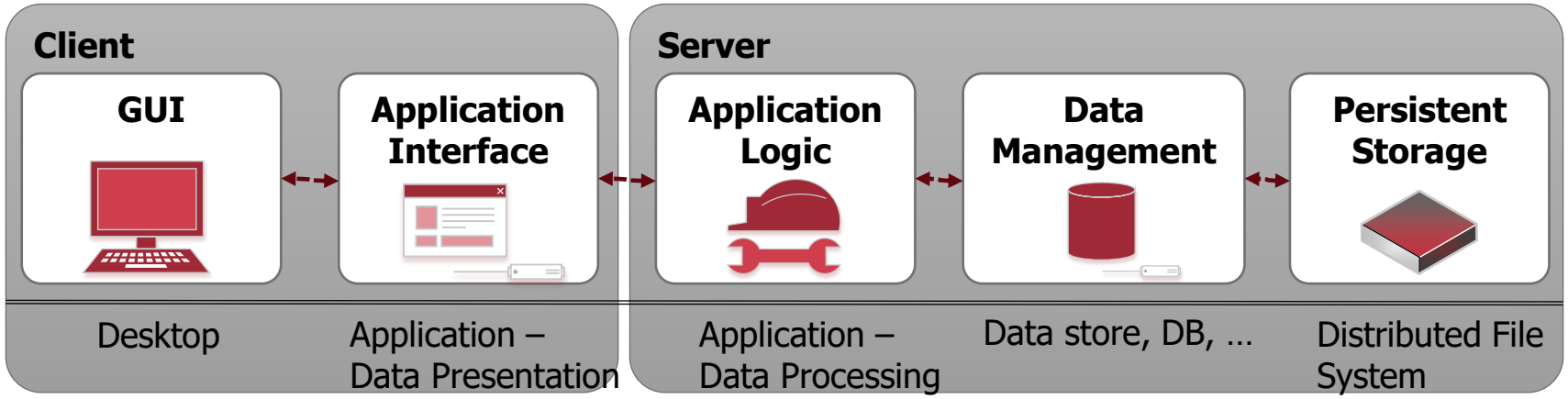
Distributing components a two-tier model results in classification of clients:

- Zero Client: Client only transmits input device events (keyboard, mouse)



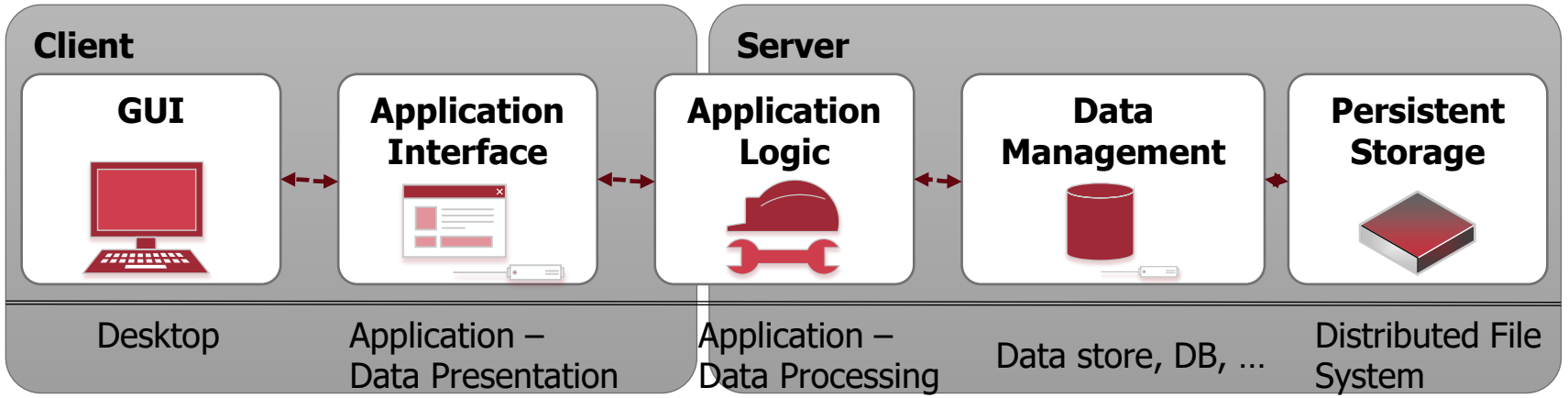
Client / Server Systems – Two Tier Model

Thin Client: responsible for presentation of result



Client / Server Systems – Two Tier Model

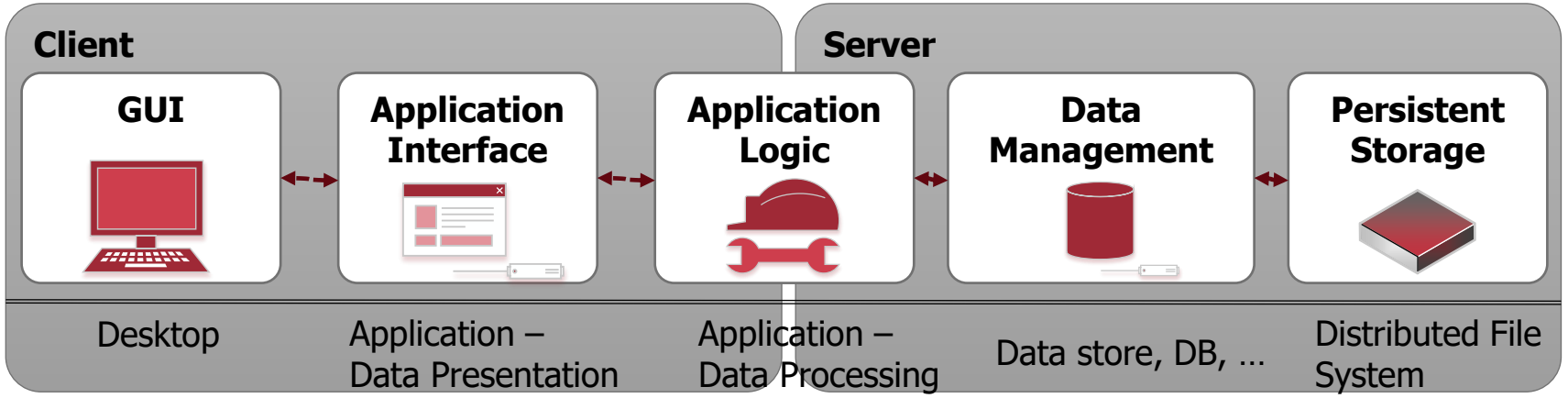
Applet Client: some processes are loaded from the server and executed locally



Client / Server Systems – Two Tier Model

Fat Client: separation within application logic → client and server cooperate

If separation exists between application logic and data management, a shared file or database server exists

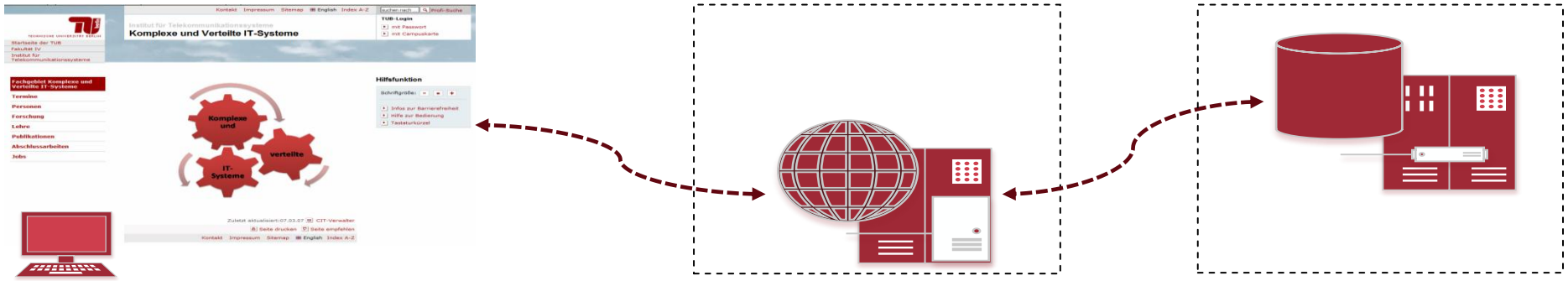


Client / Server Systems –Three Tier Model

Multi-tiered applications have advantages regarding scalability and extensibility

The three-tier model consists of:

1. Presentation of information
2. Processing of information and relations between different pieces of information
3. Storage of information



Presentation: Browser

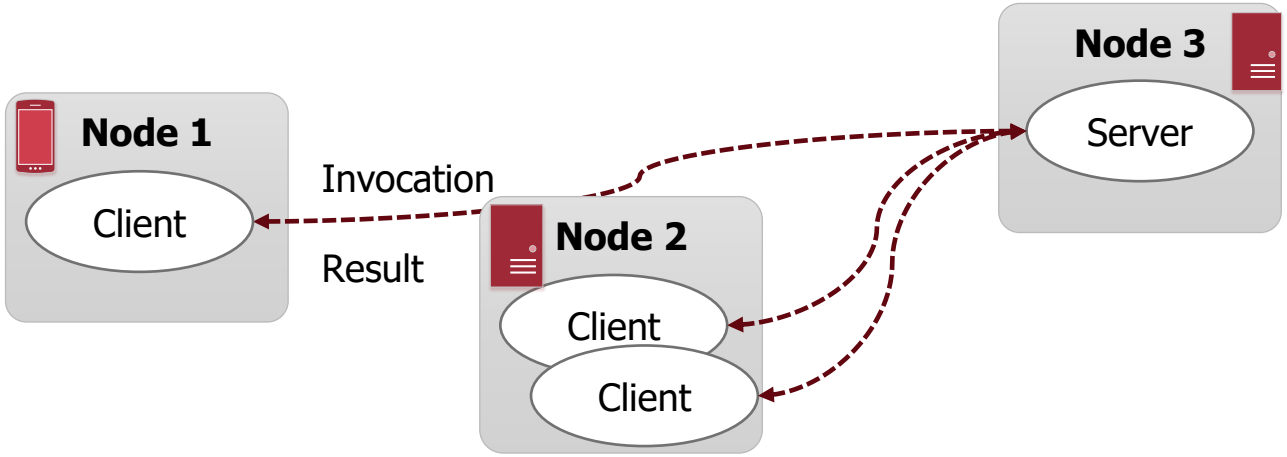
Processing: Web Server

Storage: Database

Client / Server Systems – Properties

Regular client/server systems are easy to implement

- Server as bottleneck and single point of failure (availability, reliability, performance)
- Possible improvements?
  - Parallelism within the server
  - Extended client/server systems

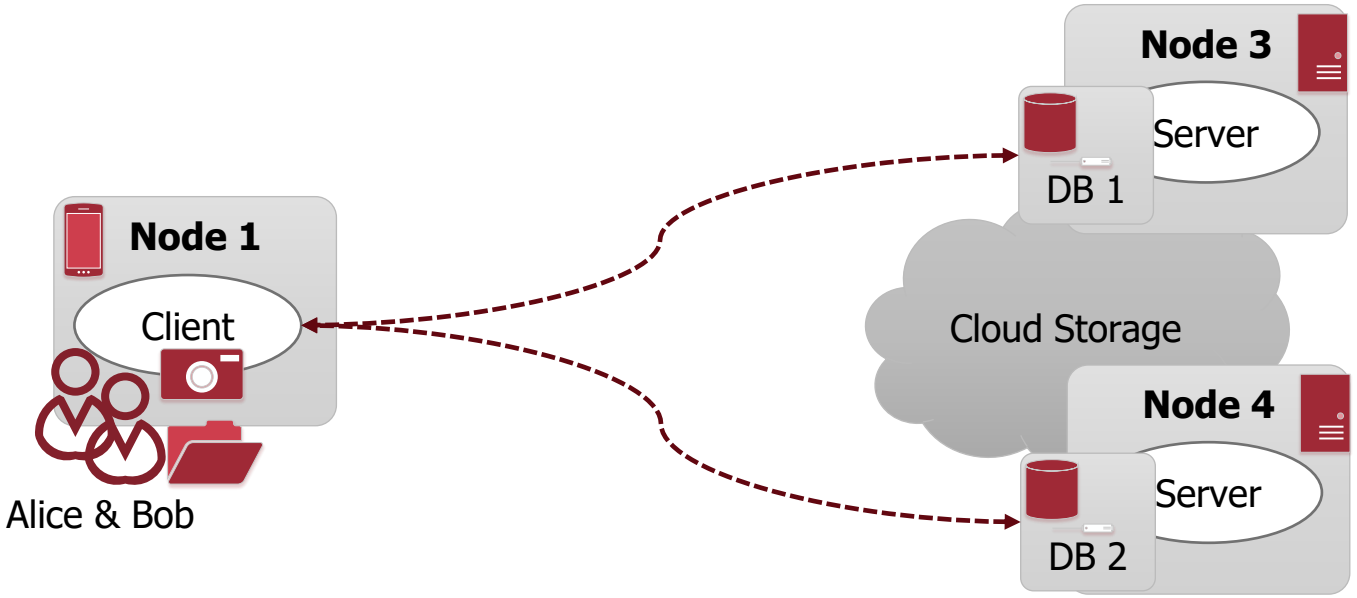


Client / Server Systems – Properties

Extended client/server systems implement services as several processes on separate computers

Possible scenarios:

- Partitioning: each server administers a part of the information, e.g. Node 3 all photos from A-M and Node 4 all photos from N-Z

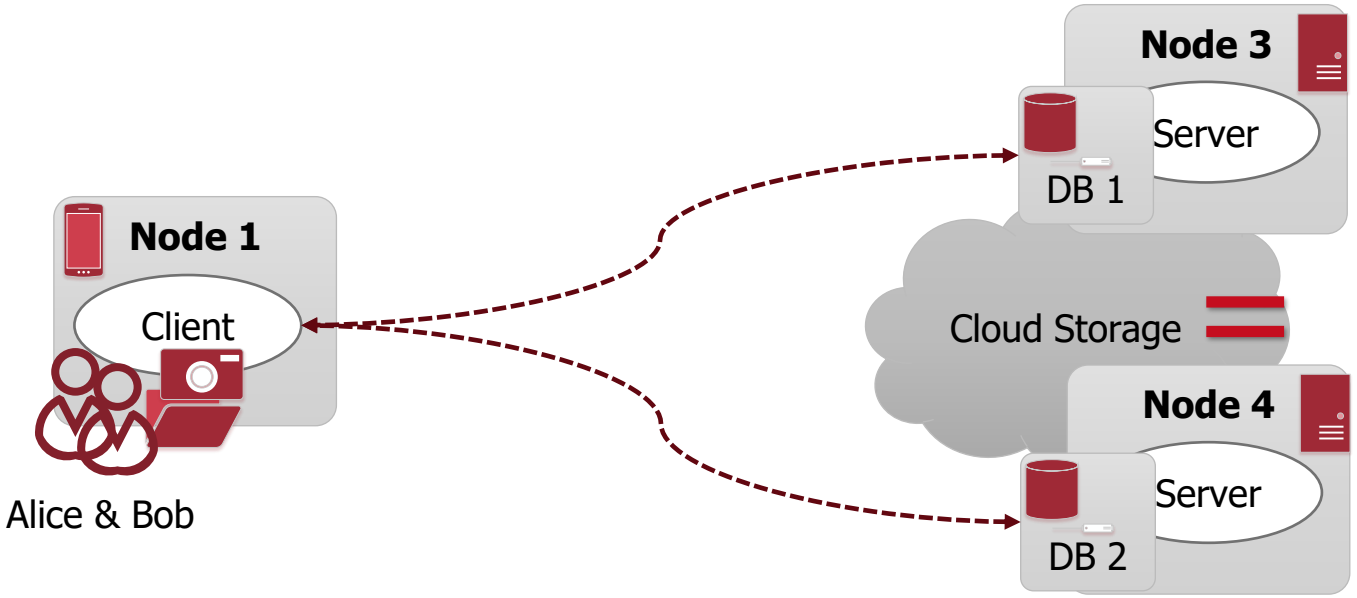


Client / Server Systems – Properties

Extended client/server systems implement services as several processes on separate computers

Possible scenarios:

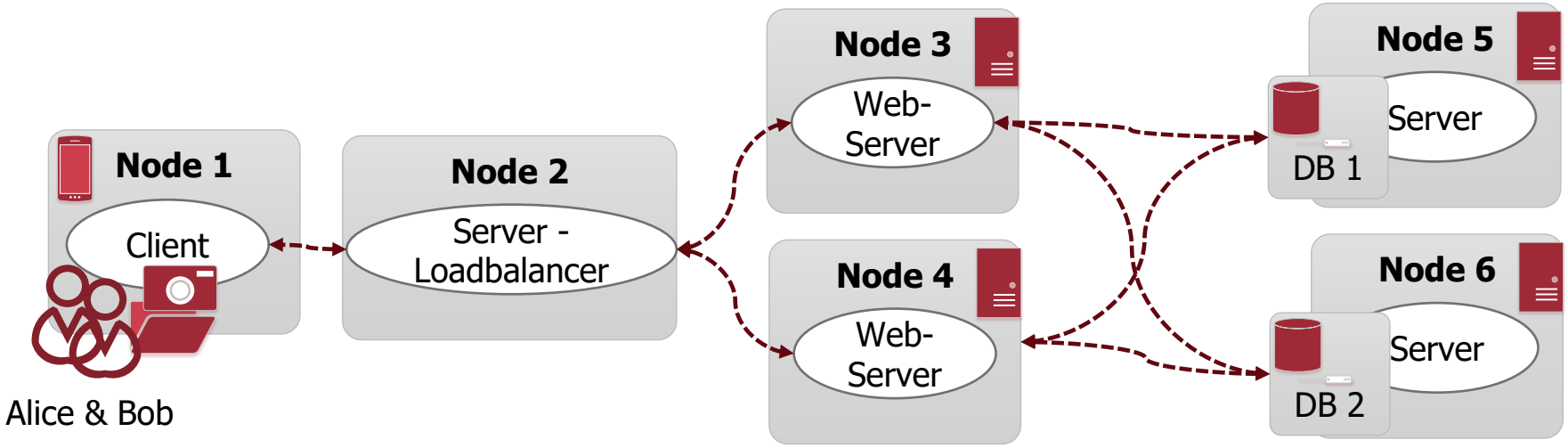
- Replication: distribution of consistent datasets on several computer, e.g. Node 3 and 4 store same set of photos to increase availability



Client / Server Systems – Properties

Extended client/server systems implement services as several processes on separate computers – possible scenarios:

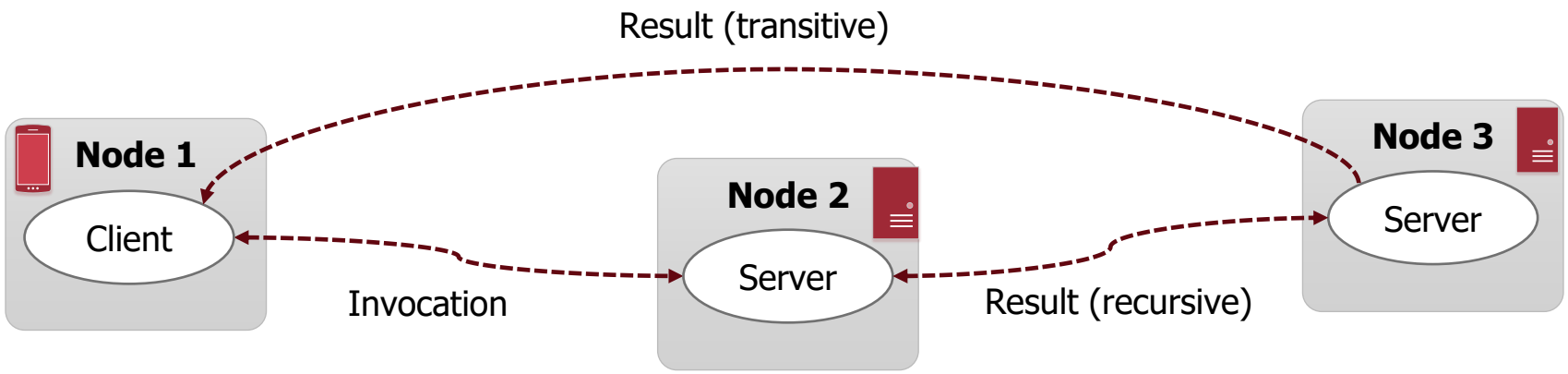
- Client-Server-Server-Systems: on-demand cooperation for service provisioning



Extended Client/Server Systems

Client-server chains can implement complex service provisioning scenarios

- Recursive: current server transmits part of the request to another server and receives results (back)
- Transitive: client gets reply from last server in the chain, not the one addressed with the request

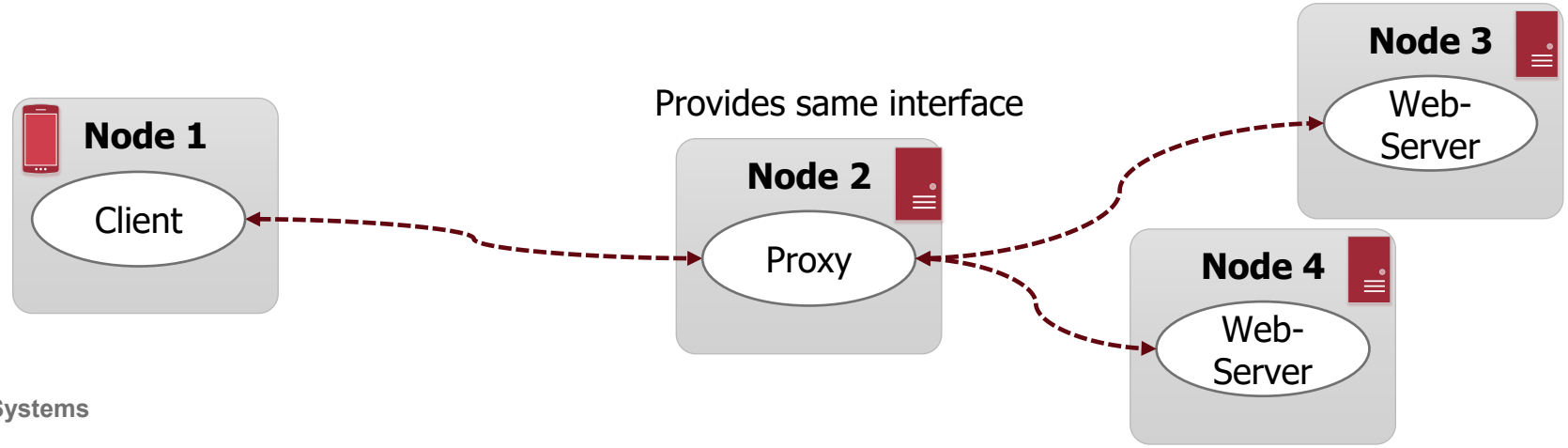


Client / Server Systems – Proxy

In-between servers can be classified regarding functionality

Proxy is delegate for several servers, widely used as cache for clients (e.g. web proxies):

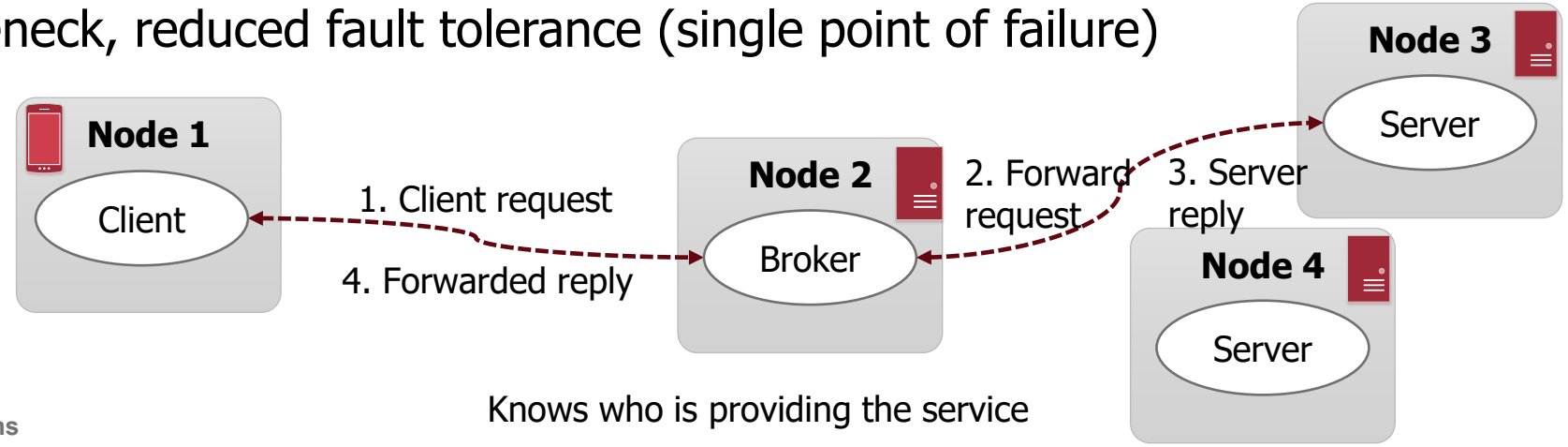
- Proxy intercepts client request and tries to serve it immediately – e.g. from the cache
- If not possible, request is forwarded to the server and result is delivered to the client
- Server results are cached and can be used to serve upcoming client requests



Client / Server Systems – Broker

Broker mediates between several clients and several servers, enables clients to access appropriate services without knowing their addresses

- All servers in the network register their services at the broker
- Broker determines matching server and delivers client request to the server
- Advantages: location and replication transparency
- Disadvantages: bottleneck, reduced fault tolerance (single point of failure)

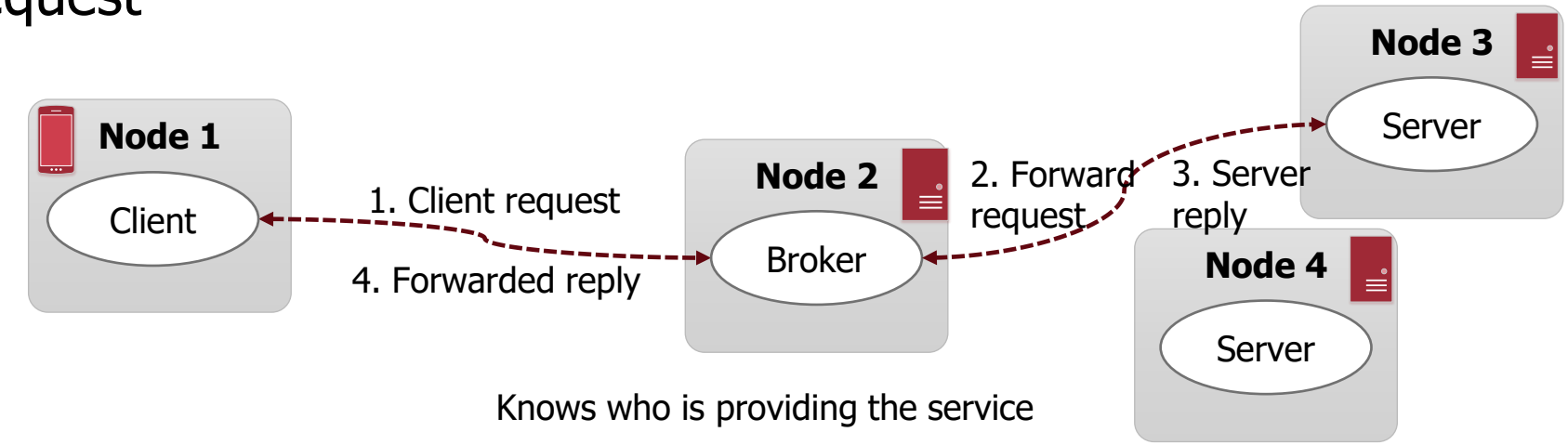


Client / Server Systems – Broker classification

Forwarding broker transmits request and response

Handle-driven broker transmits contact information of server (name, address, format of request) to client → client contacts server directly

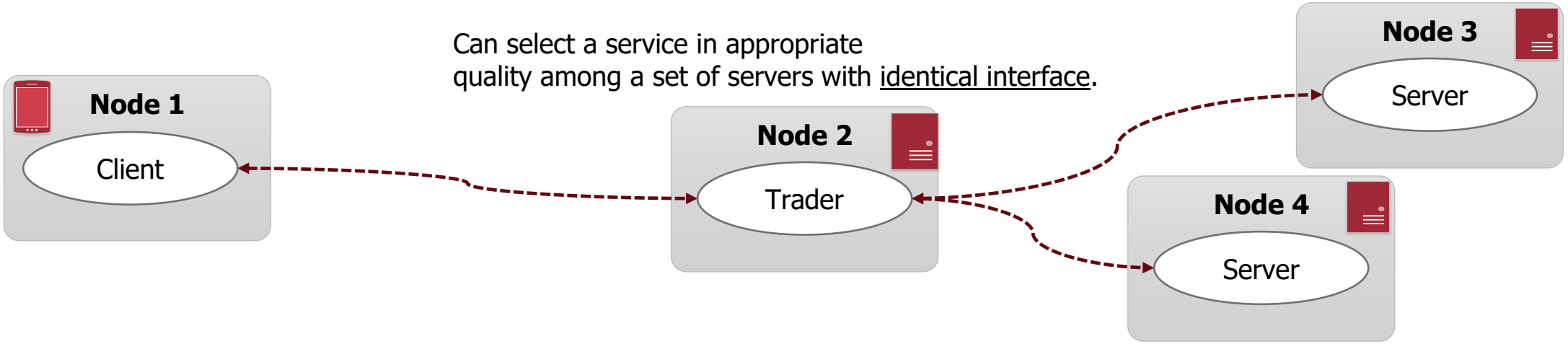
Hybrid broker supports both approaches, distinguishes according to client specification in the request



Client / Server Systems – Trader

Trader selects adequate server for the given task from a set of registered servers

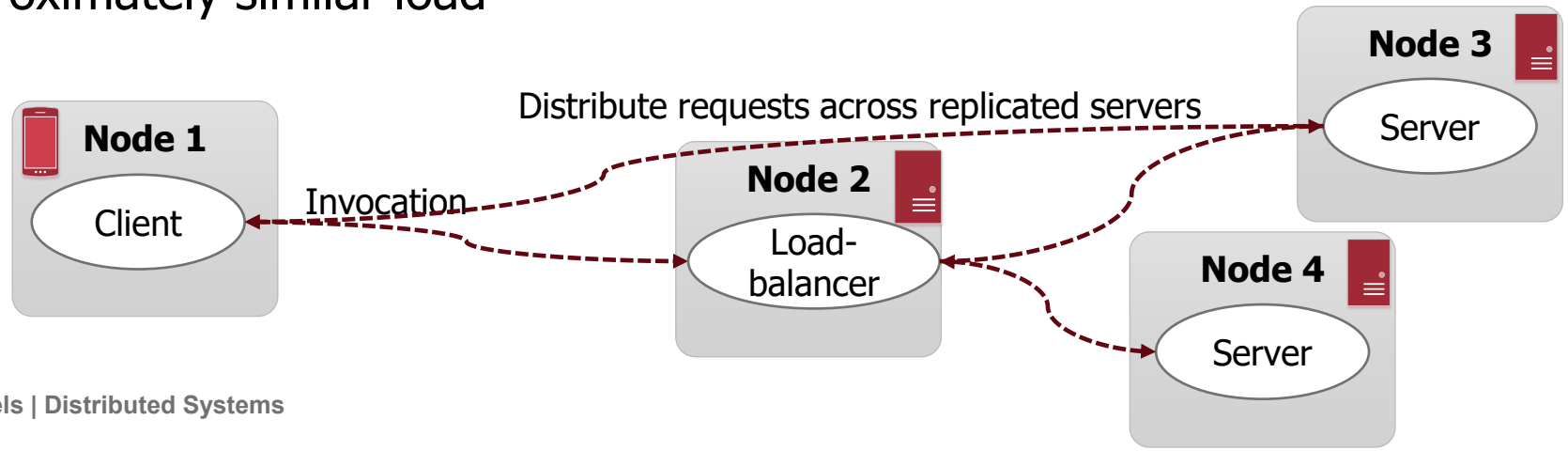
- If several servers that offer the same service in different quality are available, the trader selects the appropriate service according to client specification
- Example: videos of different qualities depending on bandwidth



Client / Server Systems – Balancer

Balancer: uniform distribution of the workload caused by the current client requests across the existing servers

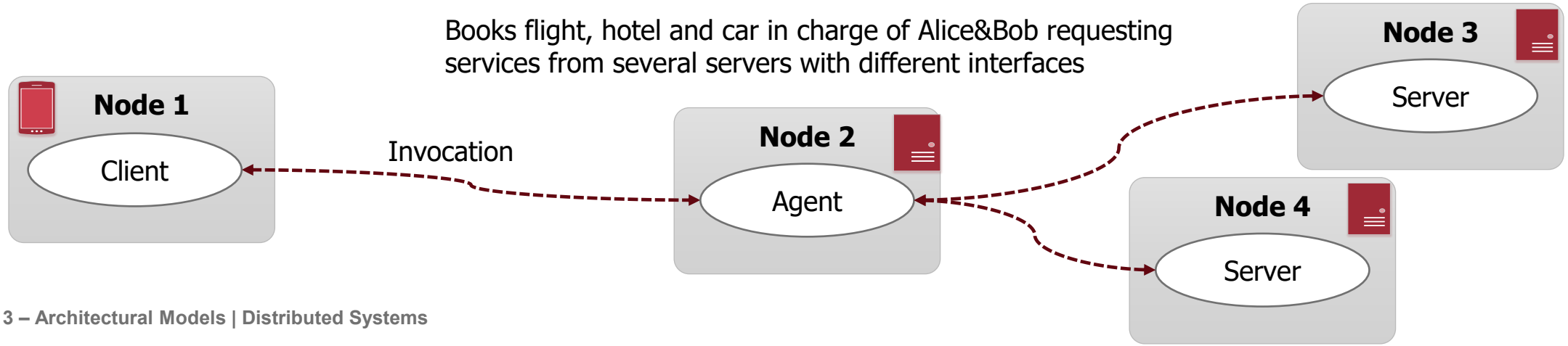
- Several identical – usually replicated – servers are available
- Balancer continuously checks and evaluates the (work)load
- Requests are distributed in a way that the response time is minimized and all servers have approximately similar load



Client / Server Systems – Agent

Agent: coordinates several server requests in charge of the steering client

- Scheduler for complex service requests
- Request is split into several parts, transmitted to different servers, responses are summarized to one response



Client / Server Systems – Interaction (Failure) Semantics

C/S entities coordinate their actions using a request/reply protocol

- Client and server process can be located on same or on different nodes
- In both cases the procedures should have the same semantics!

Coordination of interaction (remember characteristics of message passing)

- Blocking: Client is blocked and waits for reply
- Non-blocking: Client makes request and keeps on working
  - Postponed synchronous communication: response in local queue that is continuously queried by client
  - Callbacks: Upon arrival of reply, a pre-defined function triggers an interrupt
  - One way communication: no reply necessary

Client / Server Systems – Interaction (Failure) Semantics

Assumptions on failure model:

- Failures in local hardware of clients or servers (failures by crash)
- Transmission failures: Request or reply is lost or delayed
- No response → request re-send after timeout: No answer after several attempts, connection disturbed or server failure
- Several requests arrive: Client requests can change server state and affect following client requests (!)

Whether a server reaction is necessary, depends on the specific server functions

- Stateless server → no reaction necessary
- Stateful server → reaction according to selected semantic

Client / Server Systems – Interaction (Failure) Semantics

Stateless servers do not change their state due to client requests:

- Data and parameters can change, but independent of demands of the clients
- Sequence of request processing is irrelevant
- Examples: Time servers, DNS-servers, …

Stateful servers transition to a new state upon a client request:

- Depending on new state of the server certain requests cannot be satisfied
- Order of request processing has to be considered
- Example: File server (e.g. a request to delete a file changes the state)

Client / Server Systems – Interaction (Failure) Semantics

Stateful servers classified according to their communication with clients

- Stateful communication: server maintains current state using internal tables → client does not have to convey state
  - Session-based communication process
  - Internal „memory" allows server to suggest future requests and make arrangements (e.g. increase in performance)
  - In case of crash the server has to recover using the course of processing
  - Example: file server with listing of processing history
- Stateless communication

Client / Server Systems – Interaction (Failure) Semantics

Stateful servers classified according to their communication with clients

- Stateful communication
- Stateless communication: no information about past requests → client has to convey/maintain current state
  - Communication overhead, information not maintained in memory
  - Higher level of fault tolerance: Clients notice failure using timeouts and resubmit their requests

Client / Server Systems – Interaction (Failure) Semantics

Using timeouts can cause problems: Is a duplicate execution of the request tolerable, in case the message was just delayed?

Reply must follow one of the interaction semantics (determined by developer)

- At-most-once: Request is executed at most once
- At-least-once: Request is executed at least once
- Exactly-once: Request is executed exactly once
- May-be: Unreliable interaction, no guarantees are made. The message is transmitted to network without guarantee that it will arrive

Client / Server Systems – At-most-once

At-most-once: Request is executed at most once

- Message is processed only once – if at all → Duplicate filtering and retransmission of results necessary
- Server maintains a list of all received requests and corresponding replies
  - If request already in list → reply was lost, retransmit
  - If not in the list → process request, store reply in list and transmit
- When confirmed, request is deleted from the list

Client / Server Systems – At-least-once & Exactly-once

At-least-once: Request is executed at least once

- Server processes duplicate requests without check-up
- Multiple execution (request is processed at least once)
- Data inconsistency is possible

Exactly-once: Request is executed exactly once

- Practically not feasible in distributed systems
- Why?

Client / Server Systems – Interaction (Failure) Semantics

Scenario with exactly-once behavior: Client sends a print request to a server

- Server sends acknowledgement when request is received
- After recovering from a crash, the server sends “back again” to all clients

Client strategies: If client recognizes server crash, the request can be retransmitted

(1) never

(2) always

(3) only, if acknowledgement has been received

(4) only, if acknowledgement has not been received

Client / Server Systems – Exactly-once behavior

Actions taken by the server

- M send acknowledgement
- P print ticket
- C crash

Server strategies: Server can send acknowledgement…

- M → P before printing the ticket is triggered
- P → M after printing the ticket is triggered

Consider a server-crash: One possible sequence of actions is M → P → C, altogether 6 possible sequences

Client / Server Systems – Exactly-once behavior

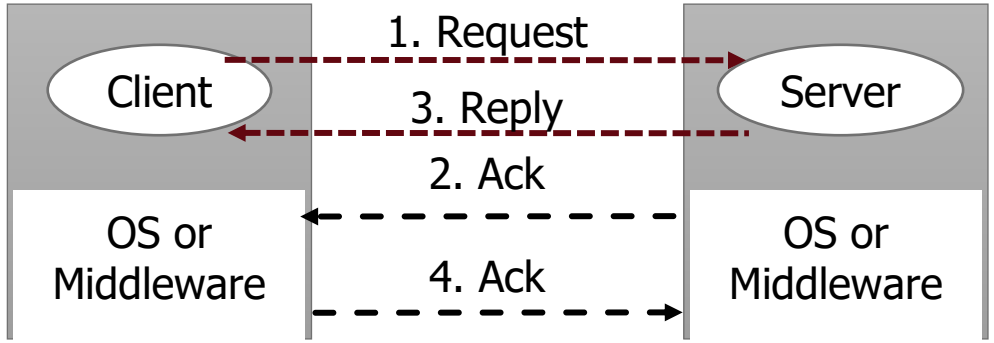
Consider combination of client and server strategies – how often is the print job executed? → no combination always successful!

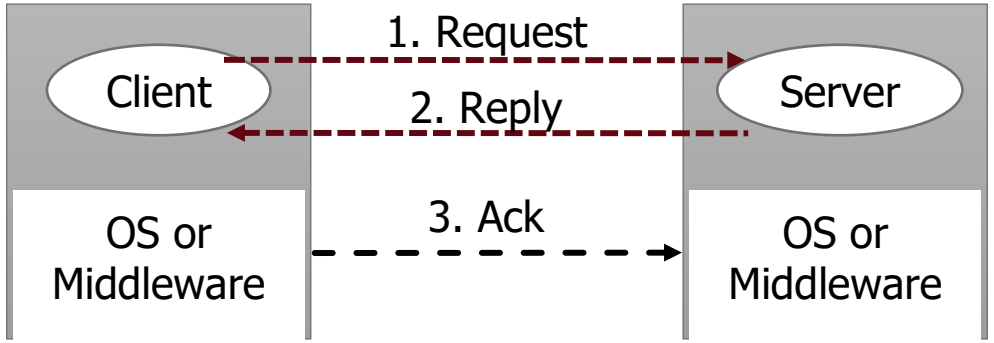
Client Retransmission	Server Strategy M → P			Server Strategy P → M		
	MPC	**MC(P)**	**C(MP)**	**PMC**	**PC(M)**	**C(PM)**
always	DUP	OK	OK	DUP	DUP	OK
never	OK	ZERO	ZERO	OK	OK	ZERO
only if ack	DUP	OK	ZERO	DUP	OK	ZERO
only if no ack	OK	ZERO	OK	OK	DUP	OK

Client / Server Systems – Interaction (Failure) Semantics

Reliable interaction in client/server systems has to be realized by the user

- Based on concepts for "retry request message", "duplicate filtering", and "retransmission of results"

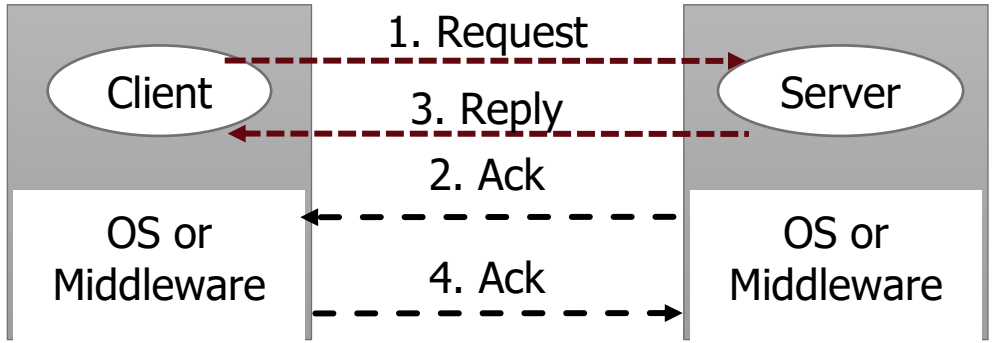


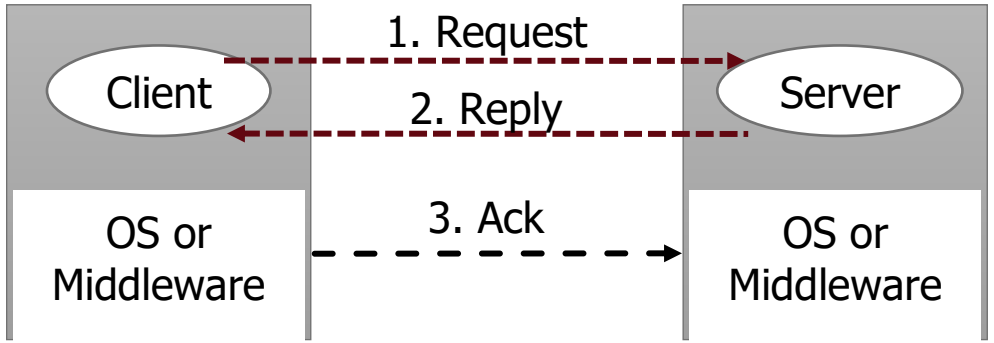


Client / Server Systems – Interaction (Failure) Semantics

Reliable interaction in client/server systems must be realized by the user

- Every message is acknowledged by sending an acknowledgment: Sender confirms receipt of request, Receiver confirms receipt of reply
- Both request and reply are acknowledged using one message: Client-server-interaction acknowledged as one unit, Client is blocked until reply arrives, then confirms reply





Questions ?